

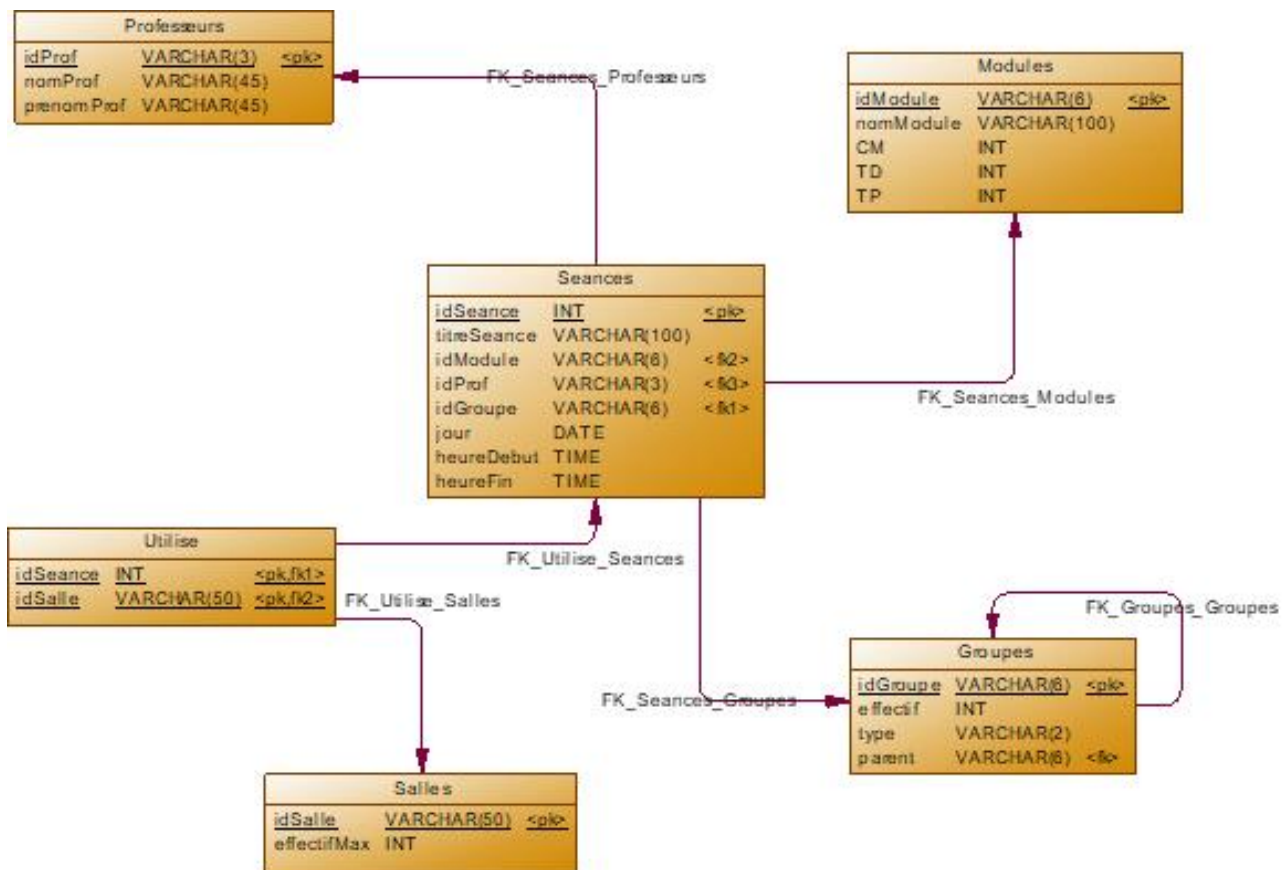
Exploitation d'une BD – Corrigé TP1 Opérateurs ensemblistes

Dans cette séance, nous écrivons des requêtes utilisant :

- des opérateurs ensemblistes :
 - l'union : UNION
 - la différence : EXCEPT
 - l'intersection : INTERSECT

- des opérateurs logiques :
 - ou logique : OR
 - et logique : AND
 - négation logique : NOT

Nous considérons la base de données **EDT** relative à la gestion des emplois du temps, dont le schéma est présenté ci-dessous :



Exercice 0 (sur feuille)

On considère les deux tables table1 et table2 suivantes :

table1

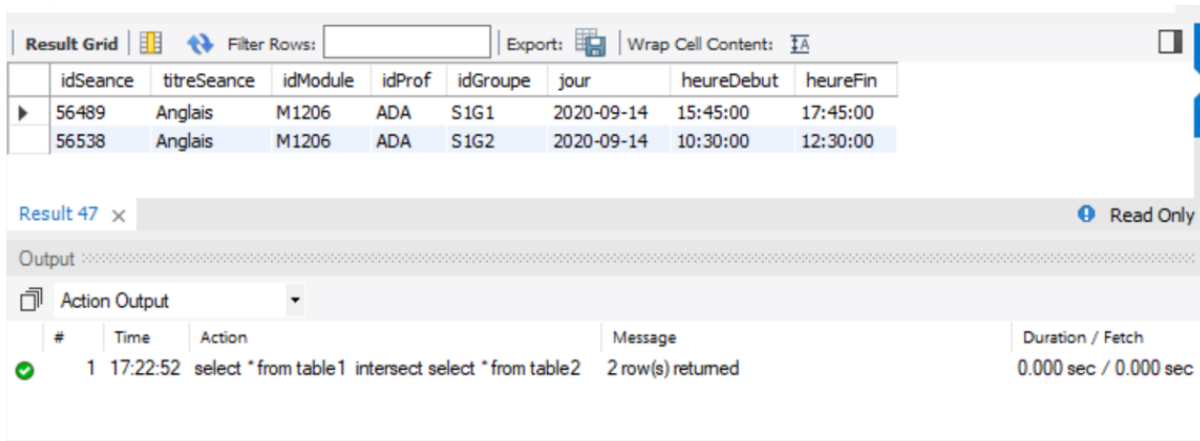
idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00

table2

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
22491	SQL	M1104	KBE	S1G3.1	2020-09-14	10:30:00	12:30:00
22492	SQL	M1104	LMO	S1G3.2	2020-09-14	10:30:00	12:30:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00
55756	SE	M1101	VOS	S1G2	2020-09-14	13:45:00	17:45:00
56488	Anglais	M1206	ADA	S1G3	2020-09-14	13:45:00	15:45:00

1. Quelles lignes va nous retourner cette requête ?

```
select * from table1
intersect
select * from table2
```



idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00

Result 47 x Read Only

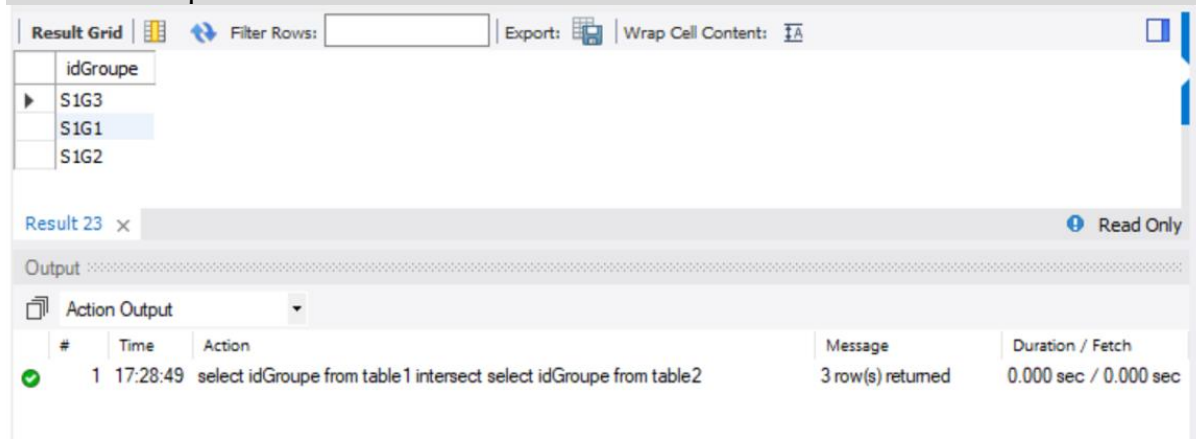
Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:22:52	select * from table1 intersect select * from table2	2 row(s) returned	0.000 sec / 0.000 sec

2. Que va nous retourner cette requête ?

```
select idGroupe from table1
intersect
select idGroupe from table2
```



idGroupe
S1G3
S1G1
S1G2

Result 23 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:28:49	select idGroupe from table1 intersect select idGroupe from table2	3 row(s) returned	0.000 sec / 0.000 sec

3. Quelles lignes va nous retourner cette requête ?

```
select * from table1
union all
select * from table2
```

Result Grid

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00
22491	SQL	M1104	KBE	S1G3.1	2020-09-14	10:30:00	12:30:00
22492	SQL	M1104	LMO	S1G3.2	2020-09-14	10:30:00	12:30:00
55756	SE	M1101	VOS	S1G2	2020-09-14	13:45:00	17:45:00
56488	Anglais	M1206	ADA	S1G3	2020-09-14	13:45:00	15:45:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56488	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00

Result 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:43:20	select * from table1 union all select * from table2	11 row(s) returned	0.000 sec / 0.000 sec

4. Combien de lignes va nous retourner cette requête ?

```
select * from table1
union distinct
select * from table2
```

Result Grid

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00
22491	SQL	M1104	KBE	S1G3.1	2020-09-14	10:30:00	12:30:00
22492	SQL	M1104	LMO	S1G3.2	2020-09-14	10:30:00	12:30:00
55756	SE	M1101	VOS	S1G2	2020-09-14	13:45:00	17:45:00
56488	Anglais	M1206	ADA	S1G3	2020-09-14	13:45:00	15:45:00

Result 13 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:46:56	select * from table1 union distinct select * from table2	9 row(s) returned	0.000 sec / 0.000 sec

5. Combien de lignes va nous retourner cette requête ?

```
select * from table1
union
select * from table2
```

Result Grid

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00
56489	Anglais	M1206	ADA	S1G1	2020-09-14	15:45:00	17:45:00
56538	Anglais	M1206	ADA	S1G2	2020-09-14	10:30:00	12:30:00
22491	SQL	M1104	KBE	S1G3.1	2020-09-14	10:30:00	12:30:00
22492	SQL	M1104	LMO	S1G3.2	2020-09-14	10:30:00	12:30:00
55756	SE	M1101	VOS	S1G2	2020-09-14	13:45:00	17:45:00
56488	Anglais	M1206	ADA	S1G3	2020-09-14	13:45:00	15:45:00

Result 14 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:48:21	select * from table1 union select * from table2	9 row(s) returned	0.015 sec / 0.000 sec

6. Quelles lignes va nous retourner cette requête ?

```
select * from table1
except
select * from table2
```

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00

Result 15 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:50:24	select * from table1 except select * from table2	3 row(s) returned	0.016 sec / 0.000 sec

7. Comment pourrions-nous écrire une autre requête que 6) avec un not in ?

```

44 #7) Comment pourrions-nous écrire une autre requête que 6) avec un not in ?
45 select *
46 from table1
47 WHERE idSeance NOT IN (
48 select idSeance from table2 );
49

```

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00
56471	Anglais	M1206	ADA	S1G3	2020-09-11	15:30:00	17:30:00

table1 17 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:53:07	select * from table1 WHERE idSeance NOT IN (select idSeance from ...	3 row(s) returned	0.000 sec / 0.000 sec

8. Que va nous retourner cette requête ?

```
select * from table1
where idGroupe not in (select idGroupe from table2)
```

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
52496	Anglais	M3206	ADA	S3G1	2020-09-11	13:30:00	15:30:00
52497	Anglais	M3206	ADA	S3G1	2020-09-10	13:45:00	15:45:00

table1 18 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:54:43	select * from table1 where idGroupe not in (select idGroupe from table2) LIMIT 0, 50000	2 row(s) returned	0.000 sec / 0.000 sec

Exercice 1 (sur feuille)

Dans la suite, on représentera : INTERSECT par $\cap$, UNION par $\cup$, EXCEPT par $\setminus$

Règles de priorité et d'associativité :

- 1) INTERSECT est prioritaire sur UNION et EXCEPT.
- 2) UNION et EXCEPT ont le même degré de priorité et sont associatifs à gauche.

Écrivez chaque égalité en rajoutant les parenthèses implicites (qui sont donc inutiles) et qui traduisent les règles ci-dessus

$$A \cap B \setminus C = (A \cap B) \setminus C$$

$$A \setminus B \cap C = A \setminus (B \cap C)$$

$$A \cup B \setminus C \cup D = ((A \cup B) \setminus C) \cup D$$

$$A \cup B \setminus C \cap D \cup E = ((A \cup B) \setminus (C \cap D)) \cup E$$

$$A \cap B \setminus C \cup D = ((A \cap B) \setminus C) \cup D$$

$$A \cap B \setminus C \cap D = (A \cap B) \setminus (C \cap D)$$

Exercice 2 (sur feuille)

On reprend la base EDT.

Soit A l'ensemble des séances où le prof ADA intervient.

Soit B l'ensemble des séances où le groupe S1G1 a cours.

Soit C l'ensemble des séances planifiées au mois d'octobre.

1. Exprimez en fonction de A, B et C ce que va nous retourner cette requête :

```
select * from Seances where idProf = "ADA"
union
select * from Seances where idGroupe = "S1G1"
intersect
select * from Seances where month(jour) = 10;
```

$$A \cup (B \cap C)$$

2. La requête suivante permet d'afficher les séances $(A \cup B) \cap C$:

```
(select * from Seances where idProf="ADA"
union
select * from Seances where idGroupe="S1G1")
intersect
select * from Seances where month(jour) = 10;
```



```

1 • (select * from Seances where idProf="ADA"
2   union
3   select * from Seances where idGroupe="S1G1")
4 ✖ intersect
5   select * from Seances where month(jour)=10;
6

```

Result 28 x

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
3879	Anglais	M3206	ADA	S3G2.2	2020-10-01	10:30:00	12:30:00
4200	Anglais	M3206	ADA	S3G3.2	2020-10-02	08:30:00	10:30:00
4370	Anglais	M3206	ADA	S3G2.1	2020-10-02	15:30:00	17:30:00
4635	Anglais	M3206	ADA	S3G1.1	2020-10-02	13:30:00	15:30:00
22405	SE	M1101	ADA	S1	2020-10-01	08:30:00	10:30:00
48207	Anglais	M1206	ADA	S1G3.2	2020-10-23	13:30:00	15:30:00

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	15:13:41	(select * from Seances where idProf="ADA" union select * from Seances where idGroupe="S1G1") intersec...	64 row(s) returned	0.016 sec / 0.000 sec

Ecrivez cette requête en utilisant une sous-requête dans la clause FROM et un alias de table dans la sous-requête.

```

32 •      select * from
33         (select * from Seances where idProf="ADA"
34         union
35         select * from Seances where idGroupe="S1G1") a
36 ✖      intersect
37         select * from Seances where month(jour)=10;
38

```

Result 13 x

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
3879	Anglais	M3206	ADA	S3G2.2	2020-10-01	10:30:00	12:30:00
4200	Anglais	M3206	ADA	S3G3.2	2020-10-02	08:30:00	10:30:00
4370	Anglais	M3206	ADA	S3G2.1	2020-10-02	15:30:00	17:30:00
4635	Anglais	M3206	ADA	S3G1.1	2020-10-02	13:30:00	15:30:00
22405	SE	M1101	ADA	S1	2020-10-01	08:30:00	10:30:00
48207	Anglais	M1206	ADA	S1G3.2	2020-10-23	13:30:00	15:30:00
50123	Anglais	M1206	ADA	S1G1.1	2020-10-21	08:30:00	10:30:00
50124	Anglais	M1206	ADA	S1G1.2	2020-10-23	08:30:00	10:30:00

Output

Action Output

#	Time	Action	Message
1	18:36:41	select * from (select * from Seances where idProf="ADA" union select * from ...	64 row(s) returned

3. Exprimez en fonction de A, B et C ce que va nous retourner la requête suivante :

```

select * from Seances where idProf="ADA"
union
select * from Seances where idGroupe="S1G1"
except
select * from Seances where month(jour) = 10 ;

```

$(A \cup B) \setminus C$

4. Ecrivez la requête retournant $(A \setminus B) \cup (B \setminus A)$. Utiliser une sous-requête dans la clause FROM et un alias de table dans la sous-requête.

```

7 select * from (select * from Seances where idProf="ADA"
8 except select * from Seances where idGroupe="S1G1") a
9 union select * from (select * from Seances where idGroupe="S1G1"
10 except select * from Seances where idProf="ADA") b;

```

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
3777	Anglais	M3206	ADA	S3G2	2020-09-25	10:30:00	12:30:00
3838	Anglais	M3206	ADA	S3G3	2020-09-25	13:30:00	15:30:00
3846	Anglais	M3206	ADA	S3G1	2020-09-22	10:30:00	12:30:00
3879	Anglais	M3206	ADA	S3G2.2	2020-10-01	10:30:00	12:30:00
4068	Anglais	M3206	ADA	S3G3.1	2020-09-30	08:30:00	10:30:00

Result 59

Output:

Action Output

#	Time	Action	Message	Duration / Fetch
15	15:46:55	select * from (select * from Seances where idProf="ADA" except select * from Seances where idGroupe="S1G1") a union select * from (select * from Seances where idGroupe="S1G1" except select * from Seances where idProf="ADA") b;	448 row(s) returned	0.016 sec / 0.000 sec

5. Ecrivez une autre requête que celle trouvée à la question précédente.

En tenant compte des règles d'associativité :

```

1 #(A \ B) U (B \ A)
2 select * from Seances where idProf="ADA"
3 except select * from Seances where idGroupe="S1G1"
4 union select * from (select * from Seances where idGroupe="S1G1"
5 except select * from Seances where idProf="ADA") a ;
6
7

```

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
3777	Anglais	M3206	ADA	S3G2	2020-09-25	10:30:00	12:30:00
3838	Anglais	M3206	ADA	S3G3	2020-09-25	13:30:00	15:30:00
3846	Anglais	M3206	ADA	S3G1	2020-09-22	10:30:00	12:30:00
3879	Anglais	M3206	ADA	S3G2.2	2020-10-01	10:30:00	12:30:00
4068	Anglais	M3206	ADA	S3G3.1	2020-09-30	08:30:00	10:30:00

Result 62

Output:

Action Output

#	Time	Action	Message	Duration / Fetch
1	15:58:50	select * from Seances where idProf="ADA" except select * from Seances where idGroupe="S1G1" union select * from (select * from Seances where idGroupe="S1G1" except select * from Seances where idProf="ADA") a;	448 row(s) returned	0.000 sec / 0.015 sec

Exercice 3.

Nous utiliserons la base de données **EDTS2** qui a le même schéma que la base de données EDT :

- Quelles sont les séances d'anglais qui ont eu lieu le 24 novembre 2020 ? Vous écrirez deux requêtes différentes : une première requête utilisant les opérateurs logiques et une deuxième utilisant les opérateurs ensemblistes.

Affichage attendu :

idSeance	titreSeance	idModule	idProf	idGroupe	jour	heureDebut	heureFin
48313	Anglais	M3206	ADA	S3G2	2020-11-24	10:30:00	12:30:00
65268	Anglais	M1206	ADA	S1G3	2020-11-24	08:30:00	10:30:00
65269	Anglais	M1206	ADA	S1G1	2020-11-24	13:45:00	15:45:00

1. Quelles sont les séances d'anglais qui ont eu lieu le 24 novembre 2020 ?

Opérateur logique

```
select *  
from Seances  
where jour="2020-11-24" and lower(titreSeance) like '%anglais';
```

Opérateur ensembliste

```
select *  
from Seances  
where jour="2020-11-24"  
intersect  
select *  
from Seances  
where lower(titreSeance) like '%anglais';
```

2. Quels sont les groupes du semestre 1 qui ont eu APE ou EDU comme enseignants ?
Vous écrirez deux requêtes différentes : une première requête utilisant uniquement des opérateurs logiques et une deuxième utilisant au moins un opérateur ensembliste. (8 lignes attendues)

Opérateur logique

```
SELECT distinct (idGroupe)  
FROM Seances  
WHERE idGroupe LIKE "S1%" AND (idProf = 'APE' OR idProf = 'EDU') ;
```

Opérateur ensembliste

```
SELECT idGroupe FROM Seances WHERE idGroupe LIKE "S1%" AND idProf = 'APE'  
union  
SELECT idGroupe FROM Seances WHERE idGroupe LIKE "S1%" AND idProf = 'EDU' ;
```

3. Quels sont les groupes du semestre 1 qui ont eu APE et EDU ? (2 lignes attendues)


```
select idGroupe from Seances where idGroupe like 'S1%'
intersect
SELECT idGroupe FROM Seances WHERE idProf = 'APE'
intersect
SELECT idGroupe FROM Seances WHERE idProf = 'EDU' ;

SELECT idGroupe FROM Seances WHERE idGroupe LIKE "S1%" AND idProf = 'APE'
intersect
SELECT idGroupe FROM Seances WHERE idGroupe LIKE "S1%" AND idProf = 'EDU' ;
```

4. Existe-il un groupe qui n'a jamais eu cours à 8:30 durant le mois de décembre ? (33 lignes attendues)

```
select idGroupe from Groupes
except
select idGroupe from Seances where heureDebut='08:30:00' AND month(jour)=12 ;

select idGroupe from Groupes where idGroupe NOT IN
(select idGroupe
from Seances
where heureDebut='08:30:00' AND month(jour)=12 );
```

5. Complétez les pointillés :

La requête ci-dessous renvoie les groupes qui **ont eu une séance le 24 novembre 2020 et qui ont (le même jour ou un autre jour) eu une séance d'anglais**. Noter que les deux conditions ne sont pas forcément sur la même séance.

```
select idGroupe from Seances where JOUR='2020-11-24'
intersect
select idGroupe from Seances where lower(titreSeance) like '%anglais%' ;
```

6. Complétez les pointillés :

La requête ci-dessous renvoie **les séances d'anglais qui ont eu lieu le 24 novembre 2020**.

```
select * from Seances where JOUR='2020-11-24'
intersect
select * from Seances where lower(titreSeance) like '%anglais%' ;
```

7. Reprenez la requête du 6 pour écrire une requête retournant les noms des groupes qui ont eu anglais le 24 novembre 2020. (3 lignes attendues)

```
select idGroupe
from (select * from Seances where JOUR='2020-11-24'
intersect
select * from Seances where lower(titreSeance) like '%anglais%') a;
```

8. Ecrivez plus simplement la requête précédente en utilisant un opérateur logique.

```
select idGroupe
from Seances
where JOUR='2020-11-24' and lower(titreSeance) like '%anglais%';
```

9. Quelles sont les salles qui n'ont pas été utilisées en décembre 2020 ? (6 lignes attendues)

```
select idSalle
from Salles
except
select idSalle FROM Utilise, Seances where Utilise.idSeance= Seances.idSeance
AND jour>='2020-12-01' AND jour<='2020-12-31' ;

select idSalle
from Salles
except
select idSalle FROM Utilise, Seances where Utilise.idSeance= Seances.idSeance
AND month(jour)=12 AND year(jour)=2020 ;
```

10. Existe-t-il une salle utilisée uniquement par Madame Anne Daumas ? (1 ligne attendue)

```
select idSalle
from Seances natural join Utilise natural join Professeurs
where nomProf="DAUMAS" AND prenomProf="ANNE"
EXCEPT
select idSalle
from Seances natural join Utilise natural join Professeurs
where nomProf<>"DAUMAS" OR prenomProf<>"ANNE" ;
```

11. La requête suivante retourne les séances en demi-groupe qui n'ont pas été planifiées le 4 mars 2021.

```
select *
from Séances
where idGroupe like "S_G_." and jour <> '2021-03-04' ;
```

- a) À quelle question répond la requête suivante ?

```
select idGroupe
from Séances
where idGroupe like "S_G_." and jour <> '2021-03-04' ;
```

Quels sont les ½ groupes qui ont eu cours un autre jour que le 4 mars 2021 ?

- b) Quels sont les demi-groupes qui n'ont pas eu cours le 4 mars 2021 ?

Ecrivez deux requêtes répondant à cette question : une avec EXCEPT et l'autre sans utiliser EXCEPT.

```
select idGroupe from Groupes where idGroupe like "S_G_._"
except
select idGroupe from Seances where idGroupe like "S_G_._" and jour='2021-03-04' ;

## Avec NOT IN, il faut rajouter la condition "IS NOT NULL"
select distinct idGroupe
from Groupes
where idGroupe like "S_G_._" and idGroupe not in
(select idGroupe from Seances where jour='2021-03-04' AND idGroupe IS NOT NULL) ;
```

12. Dites, sans l'exécuter, ce que va retourner cette requête.

```
select distinct idGroupe
from Seances
where idGroupe like "S_G_._" and idGroupe not in
(select idGroupe from Seances where jour='2021-03-03' and idGroupe IS NOT NULL
intersect
select idGroupe from Seances where jour='2021-03-04' and idGroupe IS NOT NULL) ;
```

Les demi-groupes qui n'ont pas cours les deux jours consécutifs du 3 et 4 mars 2021.

13. Vous avez vu en Maths que $\overline{A \cap B} = \overline{A} \cup \overline{B}$.

En déduire une autre requête que celle de la question précédente en utilisant une union plutôt qu'une intersection.

```
select distinct idGroupe from Seances where idGroupe like "S_G_._"
and idGroupe not in (select idGroupe from Seances where jour='2021-03-3' AND idGroupe IS NOT NULL)
union
select distinct idGroupe from Seances where idGroupe like "S_G_._"
and idGroupe not in (select idGroupe from Seances where jour='2021-03-04' AND idGroupe IS NOT NULL);
```

14. Existe-t-il une salle utilisée que pour les cours d'anglais ? (1 ligne attendue)

```
select idSalle from Utilise, Seances where Utilise.idSeance=Seances.idSeance
and lower(titreSeance) like '%anglais%'
except
select idSalle from Utilise, Seances where Utilise.idSeance=Seances.idSeance
and lower(titreSeance) not like '%anglais%' ;

## Avec NOT IN
select distinct idSalle from Utilise, Seances where Utilise.idSeance=Seances.idSeance
and lower(titreSeance) like '%anglais%' and idSalle not in (
select idSalle from Utilise, Seances where Utilise.idSeance=Seances.idSeance
and lower(titreSeance) not like '%anglais%' );
```

15. Existe-t-il une salle que Madame Anne Daumas n'a jamais utilisée ? (7 lignes attendues)

```
select idSalle from Salles
except
select idSalle
from Utilise, Seances, Professeurs
where Utilise.idSeance=Seances.idSeance and Seances.idProf=Professeurs.idProf
and nomProf = 'DAUMAS' and prenomProf='ANNE';

## Avec NOT IN
select idSalle from Salles where idSalle not in (
select idSalle
from Utilise, Seances, Professeurs
where Utilise.idSeance=Seances.idSeance and Seances.idProf=Professeurs.idProf
and nomProf = 'DAUMAS' and prenomProf='ANNE');
```

16. Quels sont les enseignants qui n'enseignent qu'en TD ("enseigner en TD" signifie que le groupe à qui le prof fait cours est du type "S_G") ? (6 lignes attendues)

```
select idProf from Professeurs
except
select idProf from Seances where idGroupe not like 'S_G';

select idProf from Professeurs
except
select idProf from Seances where (idGroupe like "S_" or idGroupe like "S_G._");
```

17. Quels sont les enseignants qui ne donnent pas de CM (càd qui n'ont jamais cours avec tous les étudiants d'un même semestre en même temps) ? (16 lignes attendues)

```
select idProf from Professeurs
except
select idProf from Seances where idGroupe like "S_";

select idProf from Professeurs where idProf not in
(select idProf from Seances where idGroupe like 'S_' and idProf IS NOT NULL) ;
```

18. Est ce qu'il y a des professeurs qui n'ont cours que le semestre 1. (4 lignes attendues)

```
select idProf from Seances
where idGroupe like 'S1%'
except
select idProf from Seances
where idGroupe not like 'S1%' ;

select idprof from Seances
except
select idprof from Seances where NOT (idGroupe like 'S1%');
```

19. Existe-il un groupe qui a cours en semaine 42 et tel que, à chaque fois qu'il a cours cette semaine-là, commence à 8h30 ?

```
select idGroupe
from Seances where week(jour)=42
except
select idGroupe
from Seances
where week(jour)=42
group by idGroupe, jour
having Min(heureDebut) > "08:30:00";
```

20. Les heures d'enseignement sont comptées de la manière suivante :

- les heures où le groupe qui a cours s'appelle S1, S2, S3 ou S4 sont des heures CM
- les heures où le groupe qui a cours s'appelle S1G1, S1G2, S1G3, S2G1, ... sont des heures TD
- les heures où le groupe qui a cours s'appelle S1G1.1, S1G1.2, S1G2.1, S1G2.2, ... sont des heures TP

Construisez la requête permettant d'afficher un récapitulatif, pour chaque prof, de son nombre d'heures CM, TD et TP.

```
select idProf, group_concat(duree) synthèse from
(select idProf, concat(' CM : ', sec_to_time(sum(time_to_sec(timediff(heurefin,heuredebut))))) duree
from Seances where idprof is not NULL and idgroupe like '___' group by idprof
union
select idProf, concat(' TD : ', sec_to_time(sum(time_to_sec(timediff(heurefin,heuredebut))))) duree
from Seances where idprof is not NULL and idgroupe like '____' group by idprof
union
select idProf, concat(' TP : ', sec_to_time(sum(time_to_sec(timediff(heurefin,heuredebut))))) duree
from Seances where idprof is not NULL and idgroupe like '_____' group by idprof) f
group by idProf;
```